AUDITORIA DE SEGURANÇA · CÓDIGO-FONTE, HISTÓRICO E BANCO

Relatório de Auditoria de Segurança — SlideAI

Data: 09/10/2026 · Commit: 4890504 (main)

Repositório: github.com/luriznx-dus-codiguin/slide-aii (público)

Valores de segredos nunca aparecem neste documento: só a impressão SHA-256 (12 caracteres).

Escopo auditado

- Código do commit 4890504 (branch main): 11 edge functions (Deno), 52 migrações SQL, 221 arquivos do front (React/Vite), scripts e documentação.
- Histórico git completo: 708 commits, todas as branches.
- Bundle do front (dist/): chaves embutidas e JWTs decodificados.
- Banco de produção (Lovable Cloud), somente leitura: RLS, políticas, grants, Storage, funções SECURITY DEFINER e triggers. Testes de lógica rodados dentro de uma transação desfeita.
- Fora do escopo: testes de intrusão em produção, dependências de terceiros (SCA) e infraestrutura da Lovable/Supabase.

Nota metodológica

Detectei a stack primeiro e adaptei cada uma das cinco categorias ao seu equivalente nela. Neste projeto, "banco sem tranca" é RLS e trigger no PostgreSQL. "Permissão no navegador" é cruzar os gates de papel do React com a RLS e as edge functions. IDOR é a checagem de posse em cada handler Deno e em cada RPC. "Chaves expostas" cobre árvore, histórico e bundle. XSS cobre os sinks do React, os renderizadores de markdown e o HTML gerado no servidor. Só entram achados verificados no código ou no banco, com arquivo e linha. O mapeamento completo está na seção 1.

15	0	1	4	6	4	13
achados	crítica	alta	média	baixa	informativa	pontos fortes

1. Stack detectada e mapeamento das categorias

Camada	Detectado
Linguagens	TypeScript (front e edge functions), SQL (PostgreSQL / PL/pgSQL)
Frontend	React 18 + Vite, Tailwind/shadcn-ui, React Router, TanStack Query, Supabase JS
Backend	11 Supabase Edge Functions (Deno) + RPCs PostgreSQL (SECURITY DEFINER)
Banco / ORM	PostgreSQL (Supabase via Lovable Cloud); migrações SQL em supabase/migrations (42) e drizzle/migrations (10, drizzle-kit); acesso via PostgREST e supabase-js — sem ORM em runtime
Autenticação	Supabase Auth (JWT); papéis em user_roles (admin, developer, user) via has_role(); OAuth para agentes MCP
Isolamento	Row Level Security (RLS) por auth.uid(); service role nas edge functions com filtro manual por user_id; triggers que travam colunas de faturamento
Integrações	Cakto (webhook de pagamento), Resend (e-mail), OpenAI e gateway de IA da Lovable, Pexels
Deploy	Lovable Cloud (sincronização com o GitHub). Não há Docker, docker-compose, CI, Helm nem Terraform no repositório

Como cada categoria foi mapeada para esta stack

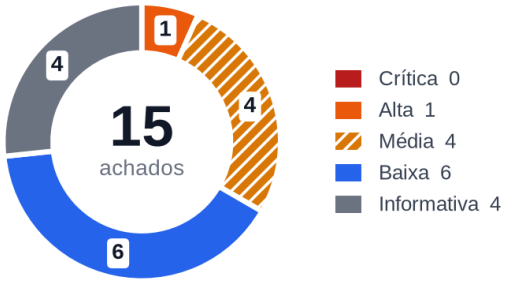
Categoria	Equivalente nesta stack e o que foi verificado
1. Banco sem tranca	O mecanismo de isolamento é RLS + triggers. Listei no banco de produção as 29 tabelas do schema public (RLS e grants por papel), as 37 políticas, os 2 buckets e 4 políticas do Storage, as 22 funções executáveis por anon/authenticated e os triggers. Nas 11 edge functions que usam service role (que ignora RLS), conferi se toda consulta filtra pelo usuário autenticado.
2. Permissão no navegador	Localizei todos os gates de papel do front (useDeveloperRole, isDeveloper, isAdmin, useEntitlement) e cruzei cada escrita das telas protegidas (AdminSupport, DevDashboard, DevModePanel, Generate) com a política RLS ou a checagem de papel da edge function correspondente. Incluí privilégios pagos decididos pelo cliente.
3. IDOR	Percorri todos os handlers das 11 edge functions (inclusive cada evento de send-email, cada fluxo de support-chat e as 5 ferramentas do MCP) e as RPCs que recebem IDs, verificando se o objeto pertence ao chamador antes de ler, alterar ou apagar.
4. Chaves expostas	git grep com 12 padrões de segredo na árvore, varredura do histórico completo (708 commits), decodificação dos JWTs do .env e do bundle (dist/), busca por defaults do tipo env ?? "literal", docs, scripts e migrações. Impressões de valores suspeitos comparadas com as registradas pelo servidor.
5. XSS / inputs sem tratamento	Front: dangerouslySetInnerHTML, innerHTML, document.write, eval/new Function, iframes, href/src dinâmicos, window.location e os 2 renderizadores de markdown (configuração de sanitização conferida no pacote). Back: HTML de e-mails (emailTemplates.ts) e da página de preview (og-preview).

Severidade: crítica = explorável hoje por anônimos com dano amplo; alta = dano financeiro ou de dados explorável por usuário comum sob condições realistas; média = exposição relevante ou com pré-condição; baixa = impacto limitado ao próprio usuário ou latente; informativa = higiene, sem exploração direta.

2. Resumo executivo

Foram 15 achados verificados: 0 crítico, 1 alto, 4 médios, 6 baixos e 4 informativos, além de 13 pontos fortes com evidência. A base é sólida: RLS em todas as tabelas, posse conferida em todas as edge functions que recebem IDs, staff verificado no servidor e nenhum segredo no código atual nem no bundle. Os riscos se concentram em colunas que o próprio dono consegue alterar (com destaque para o e-mail do perfil, que decide quem recebe um pagamento), no manuseio do segredo do webhook de pagamento e na listagem aberta de apresentações publicadas.

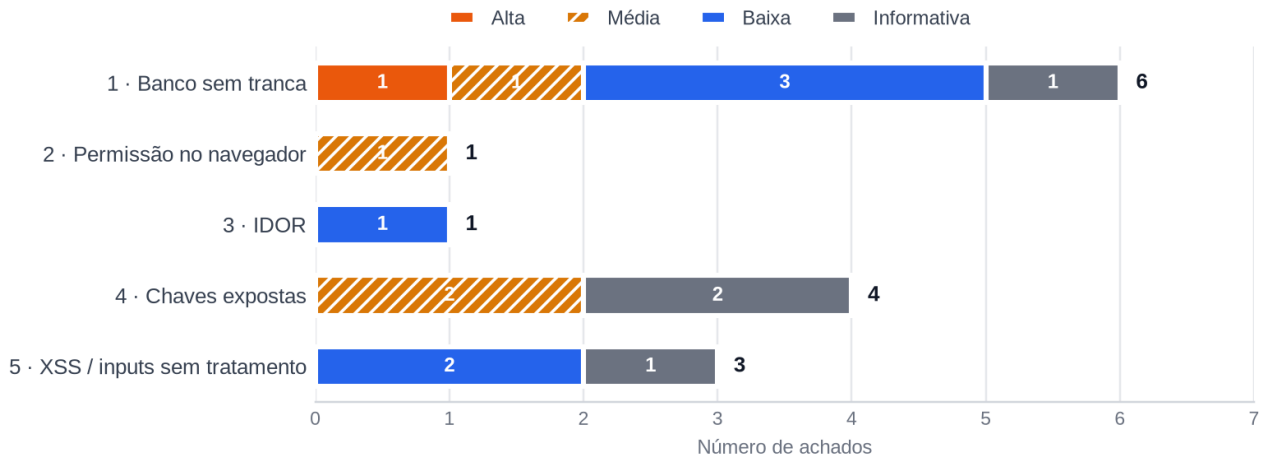
Achados por severidade



Distribuição

Severidade	Qtde.	Achados
CRÍTICA	0	—
ALTA	1	A1
MÉDIA	4	A2, A3, A4, A5
BAIXA	6	A6, A7, A8, A9, A10, A11
INFORMATIVA	4	A12, A13, A14, A15

Achados por categoria (empilhado por severidade)



A cor da severidade sempre vem com o texto. "Média" leva hachura porque é um tom vizinho de "alta".

3. Pontos fortes e pontos fracos

Pontos fortes (o que está protegido, com evidência)

	Cat.	O que está protegido	Evidência
OK	1	**RLS ligada em 29 de 29 tabelas do schema public**	Banco (pg_class.relrowsecurity). Tabelas sensíveis sem nenhuma política para clientes (só service role): billing_profiles, pending_charges, email_preferences, email_schedule, email_log, generation_attempts, edge_rate_limits, access_logs, maintenance_runs.
OK	1	**Colunas de faturamento e contadores de IA travados por trigger**	protect_billing_columns (…20260827000346…sql:206-224) e protect_ai_edit_usage (…20260927000100_platform_review_fixes.sql:233-251). handle_new_user sempre cria o perfil, o que fecha a brecha do INSERT.
OK	1	**Consultas com service role filtram pelo usuário**	assetIntelligence.ts:68 (.eq("user_id")), consume_presentation_asset e add_ai_edit_usage (WHERE user_id = _uid), generate-presentation/persist.ts (user_id do token), support-chat:171 (ocorrências do próprio usuário).
OK	3	**Toda edge function que recebe um ID confere a posse**	chat-editor/index.ts:187, regenerate-speeches/index.ts:64, support-chat/index.ts:82 e 157, send-email/index.ts:71 e 82, fetch-image (cota via RPC com _uid).
OK	2	**Toda ação de staff tem a checagem equivalente no servidor**	send-email support_reply checa papel (index.ts:52-56). As escritas do AdminSupport têm RLS has_role (catalog_admin_write, occ_admin_update, msg_staff_insert, conv_own_update, "Admin/dev gerencia artigos"). user_roles só pode ser alterada por admin.
OK	2	**Poderes de desenvolvedor são decididos no servidor**	generate-presentation/index.ts:461 e 670-674 (engineVersion e max_budget_usd só se can_user_generate = dev). fetch-image rebaixa premium para não-dev. can_user_generate força auth.uid() para clientes.
OK	3	**Webhooks e jobs sem JWT exigem segredo ou token**	cakto-webhook compara sem curto-circuito (lib.ts:85) e alerta segredo divergente; email-dispatcher confere o segredo do Vault (≥ 32 caracteres); no email-unsubscribe o token UUID é a credencial e o GET não descadastra; send-email deriva o destinatário no servidor.
OK	4	**Código atual e bundle sem segredos**	Nenhum padrão de segredo na árvore. O bundle (dist/) só tem o JWT role=anon. O front só lê VITE_SUPABASE_URL/PUBLISHABLE_KEY/PROJECT_ID, e não há defaults env ?? "literal" para segredos.
OK	5	**HTML de e-mail sempre escapado**	emailTemplates.ts:43 (escapeHtml) e :71 (inline escapa antes de aplicar **negrito**); URLs de CTA montadas no servidor. O nome no cumprimento é limitado a uma palavra só com letras (lifecycleTemplates.ts:54-61).
OK	5	**Markdown renderizado com sanitização**	Suporte (Streamdown 2.6) com rehype-sanitize + rehype-harden, que bloqueia javascript:. Central de Ajuda (react-markdown) sem HTML cru e com urlTransform padrão.
OK	5	**URLs controladas pelo usuário filtradas**	PublicProfile.tsx:28-31 (só http/https) e socialUrl; pós-login aceita só caminho relativo (Auth.tsx:41); meta tags do og-preview escapadas (esc + safeUrl).
OK	5	**Painel da equipe renderiza mensagens de usuários como texto**	AdminSupport e SupportWidget não usam HTML cru; chart.tsx (dangerouslySetInnerHTML) não é usado em lugar nenhum.
OK	3	**Proteção contra prompt injection no histórico do suporte**	support-chat/index.ts:243-245 descarta mensagens gravadas com papel system antes de chamar o modelo.

Pontos fracos (riscos centrais)

- Colunas de controle editáveis pelo dono: a RLS de UPDATE das tabelas centrais (profiles, presentations, support_*) não restringe colunas, e os triggers cobrem só parte delas (e-mail, flags pagas, papel da mensagem).
- Segredo do webhook de pagamento tratado fora do cofre: esteve em um repositório público e o valor atual foi colado no chat do projeto.
- "Publicado" significa "listável por qualquer pessoa": toda apresentação nasce publicada e a leitura pública não respeita o portfólio privado.
- Cotas de IA contadas por apresentação, uma unidade que o próprio cliente cria.

4. Achados detalhados por categoria

Para cada categoria: a tabela-resumo (Severidade | Arquivo:linha | Descrição) e, em seguida, o detalhe de cada achado (trecho de código, por que é explorável, condições, impacto e correção). Quando a categoria tem poucos achados, a cobertura verificada aparece em "Pontos fortes".

4.1 · Banco sem tranca

Severidade	Arquivo:linha	Descrição

ALTA	`supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:16`
	`supabase/migrations/20260827000346_68e8e98f-500b-4310-8345-f07ef89f505f.sql:206-224`
	`supabase/functions/cakto-webhook/index.ts:134-137`
		A1 · E-mail do perfil é editável pelo usuário e decide quem recebe o pagamento da Cakto
		profiles.email pode ser alterado pelo próprio usuário; o webhook da Cakto credita a compra ao primeiro perfil com aquele e-mail.

MÉDIA	`supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:44`
	`supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:66`
	`src/lib/legacyPersist.ts:88` · `supabase/functions/generate-presentation/persist.ts:82`
		A4 · Apresentações publicadas são listáveis por qualquer pessoa e furam o portfólio privado
		A RLS de leitura pública devolve todas as colunas de todas as apresentações publicadas, e toda apresentação gerada nasce publicada.

BAIXA	`supabase/migrations/20260716013934_90aa6f03-b7f6-4313-b724-57a99fea27f7.sql:135-137`
	`src/pages/AdminSupport.tsx:154-158`
	`supabase/functions/support-chat/index.ts:243-245`
		A6 · Usuário pode inserir mensagens com papel de "equipe" no próprio chamado
		msg_own_insert não restringe role nem metadata, e a resposta humana usa o mesmo formato.

BAIXA	`supabase/migrations/20260716013934_90aa6f03-b7f6-4313-b724-57a99fea27f7.sql:109-110`	**A7 · Dono do chamado altera qualquer coluna da conversa (estado, escalonamento, métricas)** conv_own_update não tem WITH CHECK específico nem restrição de colunas.

BAIXA	`Banco (storage.buckets)`
	`src/lib/avatarUpload.ts:3-4, 14-15, 22`
	`supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:130-140`
		A8 · Bucket de avatares sem limite de tamanho nem de tipo no servidor
		A validação de formato e tamanho existe só no navegador.

INFORMATIVA	`supabase/migrations/20260806000646_a3684224-f5cb-4467-ab32-3685b698016a.sql:23`	**A14 · Contador de visualizações pode ser inflado por anônimos** views_published_insert permite que qualquer pessoa insira visualizações de apresentações publicadas.

ALTA

A1 · E-mail do perfil é editável pelo usuário e decide quem recebe o pagamento da Cakto

Categoria 1 · Banco sem tranca · issue 1

EVIDÊNCIA

supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:16

```
CREATE POLICY "profiles_update_own" ON public.profiles FOR UPDATE USING (auth.uid() = id);
```

supabase/migrations/20260827000346_68e8e98f-500b-4310-8345-f07ef89f505f.sql:206-224

```
protect_billing_columns() trava plan, subscription_*, credits_*, cakto_* — a coluna email não está na
    lista
```

supabase/functions/cakto-webhook/index.ts:134-137

```
const { data: profile } = await admin
  .from("profiles").select("id").ilike("email", escapeLike(email)).maybeSingle();
if (profile?.id) { userId = profile.id; }
```

Banco (pg_indexes)

```
profiles_email_lower_idx ON profiles (lower(email)) — índice NÃO único
```

supabase/functions/_shared/lifecycle.ts:32 · supabase/functions/send-email/index.ts:62, 74, 86

```
let email = (p.email ?? "").trim();   // e-mails da plataforma vão para profiles.email
```

POR QUE É EXPLORÁVEL

Com a chave anon (pública) e a própria sessão, supabase.from('profiles').update({ email: 'vitima@empresa.com' }).eq('id', meuId) passa pela RLS e pelo trigger. Quando a vítima compra com esse e-mail, o webhook encontra o perfil do atacante e libera nele o plano ou os créditos.

CONDIÇÕES DE EXPLORAÇÃO

O e-mail da compra não pode estar em outro perfil: com dois perfis iguais o maybeSingle falha e o webhook cai em auth.users, que resolve a vítima. Vale para quem compra antes de criar a conta ou com outro e-mail. É um ataque direcionado: o atacante precisa saber o e-mail com antecedência.

IMPACTO

Desvio de compras e assinaturas (prejuízo direto ao cliente e chargeback para o SlideAI). Os e-mails da conta do atacante (boas-vindas, nutrição, suporte, portfólio) passam a ser enviados para um endereço escolhido por ele, usando o domínio slideai.com.br.

CORREÇÃO SUGERIDA

Incluir email no trigger de colunas protegidas (ou sincronizar profiles.email a partir de auth.users por trigger). No webhook, resolver primeiro por auth.users (find_user_id_by_email) e só depois pelo perfil. Criar índice único em lower(email).

MÉDIA

A4 · Apresentações publicadas são listáveis por qualquer pessoa e furam o portfólio privado

Categoria 1 · Banco sem tranca · issue 3

EVIDÊNCIA

supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:44

```
CREATE POLICY "presentations_public_read" ON public.presentations FOR SELECT
  USING (is_published = true AND deleted_at IS NULL);
```

supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:66

```
CREATE POLICY "slides_public_read" ON public.slides FOR SELECT …
```

src/lib/legacyPersist.ts:88 · supabase/functions/generate-presentation/persist.ts:82

```
is_paid: true, is_published: true,
```

supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:61-65

```
get_profile_for_viewer(_username) RETURNS TABLE (id uuid, …) — devolve o id mesmo sem acesso
```

supabase/functions/mcp/index.ts:81 · src/lib/mcp/tools/list-presentations.ts:22-23

```
supabase.from("presentations").select("id, title, slug, …")  // sem filtro por user_id
```

POR QUE É EXPLORÁVEL

Com a chave anon: GET /rest/v1/presentations?select=*&is_published=eq.true&user_id=eq.<id> lista todas as apresentações de um portfólio privado, incluindo creative_brief, brand_identity e presenters_names. A verificação can_view_portfolio só existe na RPC get_portfolio_presentations. A ferramenta list_presentations do MCP também mistura as apresentações publicadas de todos os usuários.

CONDIÇÕES DE EXPLORAÇÃO

Basta a chave anon pública. Em produção, 50 de 52 apresentações estão publicadas, hoje todas de uma única conta.

IMPACTO

Exposição de conteúdo e de metadados de marca de todos os usuários e quebra da promessa de portfólio privado com pedido de acesso.

CORREÇÃO SUGERIDA

Criar apresentações como não publicadas por padrão. Servir a leitura pública por slug via RPC ou view com colunas mínimas, sem listagem aberta. Fazer a listagem por dono respeitar can_view_portfolio e filtrar por user_id no MCP.

BAIXA A6 · Usuário pode inserir mensagens com papel de "equipe" no próprio chamado

Categoria 1 · Banco sem tranca · issue 5

EVIDÊNCIA

supabase/migrations/20260716013934_90aa6f03-b7f6-4313-b724-57a99fea27f7.sql:135-137

```
CREATE POLICY "msg_own_insert" ON public.support_messages FOR INSERT TO authenticated
  WITH CHECK (EXISTS (… c.user_id = auth.uid()));
```

src/pages/AdminSupport.tsx:154-158

```
insert({ conversation_id, role: "assistant", content, metadata: { source: "human_agent" } })
```

supabase/functions/support-chat/index.ts:243-245

```
.filter((m) => m.role === "user" || m.role === "assistant")   // assistant forjado vai ao modelo
```

POR QUE É EXPLORÁVEL

Via PostgREST, o dono do chamado insere role="assistant" e metadata.source="human_agent" (por exemplo, "reembolso aprovado"). A mensagem aparece como resposta da equipe e entra no contexto do modelo.

CONDIÇÕES DE EXPLORAÇÃO

Afeta só o próprio chamado.

IMPACTO

Engenharia social contra a equipe de suporte e indução do assistente de IA.

CORREÇÃO SUGERIDA

Forçar role='user' e metadata vazia no INSERT feito pelo dono (WITH CHECK ou trigger), deixando role assistant/system só para staff e service role.

BAIXA A7 · Dono do chamado altera qualquer coluna da conversa (estado, escalonamento, métricas)

Categoria 1 · Banco sem tranca · issue 5

EVIDÊNCIA

supabase/migrations/20260716013934_90aa6f03-b7f6-4313-b724-57a99fea27f7.sql:109-110

```
CREATE POLICY "conv_own_update" ON public.support_conversations FOR UPDATE TO authenticated
  USING (user_id = auth.uid() OR has_role(…admin) OR has_role(…developer));
```

POR QUE É EXPLORÁVEL

O usuário grava state='escalated', escalated_at, resolved_by_human, reopen_count ou rating direto pela API.

CONDIÇÕES DE EXPLORAÇÃO

Afeta só as conversas do próprio usuário.

IMPACTO
Furar a fila de atendimento humano e distorcer as métricas de suporte.

CORREÇÃO SUGERIDA
Remover o UPDATE do dono, porque as mudanças de estado já passam pela edge function support-chat, ou travar por trigger as colunas de staff.

BAIXA

A8 · Bucket de avatares sem limite de tamanho nem de tipo no servidor

Categoria 1 · Banco sem tranca · issue 6

EVIDÊNCIA
`Banco (storage.buckets)`

```
avatars: file_size_limit = NULL, allowed_mime_types = NULL
```

`src/lib/avatarUpload.ts:3-4, 14-15, 22`

```
const ALLOWED = ["image/png", …]; const MAX_BYTES = 4 * 1024 * 1024;
.upload(path, file, { upsert: true, contentType: file.type, … })
```

`supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:130-140`

```
avatars_read_auth (anon, authenticated: todo o bucket); avatars_insert_own / update_own (pasta do
    usuário)
```

POR QUE É EXPLORÁVEL
Chamando a API do Storage direto, o usuário sobe arquivos de qualquer tipo e tamanho (HTML, SVG, binários grandes) na própria pasta. Eles ficam acessíveis por URL assinada de 10 anos e podem ser lidos por anônimos.

CONDIÇÕES DE EXPLORAÇÃO
Basta uma conta autenticada.

IMPACTO
Hospedagem de conteúdo arbitrário no domínio de storage do projeto e custo de armazenamento.

CORREÇÃO SUGERIDA
Definir file_size_limit (4 MB) e allowed_mime_types (png, jpeg, webp, gif) no bucket.

INFORMATIVA

A14 · Contador de visualizações pode ser inflado por anônimos

Categoria 1 · Banco sem tranca · issue 7

EVIDÊNCIA
`supabase/migrations/20260806000646_a3684224-f5cb-4467-ab32-3685b698016a.sql:23`

```
CREATE POLICY views_published_insert ON public.slide_views … (p.is_published = true OR p.user_id =
    auth.uid())
```

POR QUE É EXPLORÁVEL
Um script com a chave anon insere linhas em massa, e o trigger bump_presentation_view_count infla view_count.

CONDIÇÕES DE EXPLORAÇÃO
Basta a chave anon.

IMPACTO
Métricas falsas e crescimento da tabela.

CORREÇÃO SUGERIDA
Registrar visualizações por RPC com deduplicação (sessão ou IP com hash, por janela de tempo).

4.2 · Permissão no navegador

Severidade	Arquivo:linha	Descrição
MÉDIA	src/lib/legacyPersist.ts:86-90 supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:43 supabase/migrations/20260927000100_platform_review_fixes.sql:233-251	**A5 · Recurso pago (falas) e cotas de IA por apresentação dependem de dados que o próprio cliente grava** O servidor confia em include_speeches, gravado pelo navegador, e as cotas são contadas por apresentação, que o cliente pode criar à vontade.

MÉDIA

A5 · Recurso pago (falas) e cotas de IA por apresentação dependem de dados que o próprio cliente grava

Categoria 2 · Permissão no navegador · issue 4

EVIDÊNCIA

src/lib/legacyPersist.ts:86-90

```
supabase.from("presentations").insert({ …, is_paid: true, is_published: true,
  presenters_names: form.presentersNames, include_speeches: form.includeSpeeches, … })
```

supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:43

```
presentations_owner_all … FOR ALL USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id)
```

supabase/migrations/20260927000100_platform_review_fixes.sql:233-251

```
protect_ai_edit_usage() protege só ai_edit_messages, ai_edit_complex e speech_regen_count
```

supabase/functions/regenerate-speeches/index.ts:16, 67, 88

```
export const MAX_SPEECH_REGENS = 5;
if (!(pres as any).include_speeches) { … 402 }
const targets = scope === "all" ? slideRows : …   // sem teto de slides
```

supabase/functions/chat-editor/index.ts:198-211

```
Cota de edição por apresentação (10 mensagens ou 3 edições complexas)
```

POR QUE É EXPLORÁVEL

Um usuário com plano ativo (ou bônus) cria pela API apresentações com include_speeches=true (sem pagar os +50 créditos) e com quantos slides quiser, depois chama regenerate-speeches até 5 vezes em cada uma. Cada apresentação nova também zera a cota do chat-editor, cujo limite real é o rate limit de 40/h.

CONDIÇÕES DE EXPLORAÇÃO

Exige conta autenticada com direito de uso (can_user_generate com custo 0).

IMPACTO

Custo de IA sem cobrança e contorno do preço das falas.

CORREÇÃO SUGERIDA

Travar include_speeches, is_paid e presenters_* por trigger (só service role grava), ou fazer a edge function de geração criar o registro. Contar as cotas por usuário e período, e limitar o número de slides em regenerate-speeches.

4.3 · IDOR

Severidade	Arquivo:linha	Descrição
BAIXA	supabase/migrations/20260806000729_867c7dc3-03da-4aea-aa55-d62f0694eb74.sql:38 supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:41, 120 supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:61-65, 121	**A9 · RPCs liberadas para anônimos revelam papéis e relações de acesso de terceiros** has_role, can_view_portfolio e get_profile_for_viewer aceitam IDs arbitrários sem checar o chamador.

BAIXA

A9 · RPCs liberadas para anônimos revelam papéis e relações de acesso de terceiros

Categoria 3 · IDOR · issue 7

EVIDÊNCIA

`supabase/migrations/20260806000729_867c7dc3-03da-4aea-aa55-d62f0694eb74.sql:38`

```
GRANT EXECUTE ON FUNCTION public.has_role(uuid, app_role) TO anon, authenticated, service_role;
```

`supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:41, 120`

```
can_view_portfolio(_owner uuid, _viewer uuid) … GRANT EXECUTE … TO anon, authenticated;
```

`supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:61-65, 121`

```
get_profile_for_viewer(_username) devolve id, created_at, is_public de qualquer username
```

POR QUE É EXPLORÁVEL

Com IDs obtidos em presentations.user_id (público) ou em get_profile_for_viewer, um anônimo descobre quais contas são admin ou developer (has_role) e quem tem acesso aprovado a um portfólio privado (can_view_portfolio com _viewer arbitrário).

CONDIÇÕES DE EXPLORAÇÃO

Basta a chave anon.

IMPACTO

Enumeração de alvos privilegiados e vazamento de relações entre usuários.

CORREÇÃO SUGERIDA

Revogar EXECUTE de anon e authenticated em has_role (as políticas o chamam como definer) e em can_view_portfolio, ou fazer can_view_portfolio usar só auth.uid().

4.4 · Chaves expostas

Severidade	Arquivo:linha	Descrição
MÉDIA	`commit 1cdc8b0 (01/08/2026) · supabase/functions/cakto-webhook/index.test.ts:13` `GitHub API (repos/Iuriznx-dus-codigui n/slide-aii)`	**A2 · Segredo de produção do webhook da Cakto ficou no histórico git de um repositório público** Commit de 01/08/2026 trouxe o segredo real em um teste; foi trocado no dia seguinte, mas continua no histórico. O segredo só foi rotacionado em 28/09.
MÉDIA	`Histórico de mensagens do projeto na Lovable (29/09/2026 00:43 UTC)`	**A3 · Segredo atual do webhook da Cakto foi colado em texto puro no chat do projeto na Lovable** Achado fora do repositório, verificado pela API da Lovable. O valor enviado no chat em 29/09/2026 tem a impressão do segredo em uso.
INFORMATIVA	`.env:1-3` `.env.example:1-6`	**A12 · .env versionado com a chave pública (anon) do Supabase** A chave é pública por design (role=anon), mas o .env real está no git e fora do .gitignore.
INFORMATIVA	`supabase/migrations/20260517133059_9502e9f2-1012-4cec-8021-84c751280b72.sql:38-41`	**A13 · Migração concede admin e developer a um UUID fixo** Credencial de privilégio embutida no código de um repositório público.

MÉDIA

A2 · Segredo de produção do webhook da Cakto ficou no histórico git de um repositório público

Categoria 4 · Chaves expostas · issue 2

EVIDÊNCIA

`commit 1cdc8b0 (01/08/2026) · supabase/functions/cakto-webhook/index.test.ts:13`

```
const SECRET = "…[valor oculto]";   // removido em eec60fc (02/08/2026), segue no histórico
```

Impressão SHA-256[:12] do valor

```
46925c8066f7 — igual à impressão esperada registrada pelo servidor (security_events) até 28/09/2026. A
    esperada hoje é 2bf40c2c9acd.
```

GitHub API (repos/Iuriznx-dus-codiguin/slide-aii)

```
"private": false, "visibility": "public"
```

POR QUE É EXPLORÁVEL

Entre 01/08 e 28/09/2026, qualquer pessoa com o histórico podia enviar eventos falsos (purchase_approved, subscription_created…) com o segredo válido e liberar planos ou créditos para qualquer e-mail.

CONDIÇÕES DE EXPLORAÇÃO

Hoje o valor não é mais aceito: o segredo foi rotacionado e a lista esperada só tem 2bf40c2c9acd. Forense: os 7 eventos de pagamento entre 01/08 e 29/09 não foram creditados a nenhuma conta (user_id nulo).

IMPACTO

Risco residual baixo enquanto o segredo antigo não voltar a ser usado. A severidade era crítica durante a janela de exposição.

CORREÇÃO SUGERIDA

Manter o segredo antigo fora de qualquer lista aceita, confirmar no painel da Cakto, usar só valores fictícios em testes e adicionar varredura de segredos (gitleaks) no fluxo de PR. Reescrever o histórico é opcional, porque o valor já foi rotacionado.

MÉDIA

A3 · Segredo atual do webhook da Cakto foi colado em texto puro no chat do projeto na Lovable

Categoria 4 · Chaves expostas · issue 2

EVIDÊNCIA

Histórico de mensagens do projeto na Lovable (29/09/2026 00:43 UTC)

```
configure o webhook cakto … segredo: …[valor oculto]
```

Impressão SHA-256[:12]

```
2bf40c2c9acd — igual à impressão esperada atual do cakto-webhook
```

POR QUE É EXPLORÁVEL

Quem tem acesso ao histórico do projeto (membros do workspace; visibilidade workspace_edit) ou ao armazenamento do fornecedor pode forjar eventos de pagamento válidos.

CONDIÇÕES DE EXPLORAÇÃO

É preciso ter acesso ao projeto na Lovable. O segredo continua válido.

IMPACTO

Liberação indevida de planos e créditos e eventos de reembolso falsos.

CORREÇÃO SUGERIDA

Rotacionar o segredo na Cakto e no secret CAKTO_WEBHOOK_SECRET. Cadastrar o novo valor só pela tela de Secrets, nunca no chat.

INFORMATIVA

A12 · .env versionado com a chave pública (anon) do Supabase

Categoria 4 · Chaves expostas · issue 9

EVIDÊNCIA

.env:1-3

```
VITE_SUPABASE_PROJECT_ID / VITE_SUPABASE_PUBLISHABLE_KEY (JWT role=anon) / VITE_SUPABASE_URL
```

.env.example:1-6

```
"o repositório não deveria versionar um .env com valores reais preenchidos"
```

POR QUE É EXPLORÁVEL

Não é um segredo, porque a segurança vem da RLS. O risco é alguém acrescentar uma chave real (service role, OpenAI) a esse arquivo já rastreado.

CONDIÇÕES DE EXPLORAÇÃO

—

IMPACTO
Higiene de configuração.

CORREÇÃO SUGERIDA
Adicionar .env ao .gitignore e manter só o .env.example. Lembrar que a Lovable injeta as variáveis no build.

INFORMATIVA A13 · Migração concede admin e developer a um UUID fixo

Categoria 4 · Chaves expostas · issue 9

EVIDÊNCIA
`supabase/migrations/20260517133059_9502e9f2-1012-4cec-8021-84c751280b72.sql:38-41`

```
INSERT … VALUES ('54783c81-…', 'developer'), ('54783c81-…', 'admin') ON CONFLICT DO NOTHING;
```

POR QUE É EXPLORÁVEL
Expõe o ID da conta administradora, facilitando ataques direcionados junto com A9. Todo ambiente criado a partir do repositório herda esse admin.

CONDIÇÕES DE EXPLORAÇÃO
—

IMPACTO
Higiene e alvo conhecido.

CORREÇÃO SUGERIDA
Conceder papéis por procedimento operacional (SQL manual documentado), não por migração.

4.5 · XSS / inputs sem tratamento

Severidade	Arquivo:linha	Descrição
BAIXA	`src/components/PresenterNotesPanel.tsx:86-114`	**A10 · Roteiro de falas é escrito com document.write sem escapar HTML** Título, headlines, nomes e falas entram crus numa janela do mesmo domínio.
BAIXA	`supabase/functions/og-preview/index.ts:63-70`	**A11 · og-preview insere título e descrição num bloco <script> JSON-LD sem escapar "<"** JSON.stringify não neutraliza </script>; as meta tags estão escapadas, mas o JSON-LD não.
INFORMATIVA	`src/pages/PublicProfile.tsx:170`	**A15 · avatar_url aceita qualquer URL externa e é carregada por quem visita o perfil** O usuário grava a URL que quiser em profiles.avatar_url.

BAIXA A10 · Roteiro de falas é escrito com document.write sem escapar HTML

Categoria 5 · XSS / inputs sem tratamento · issue 8

EVIDÊNCIA
`src/components/PresenterNotesPanel.tsx:86-114`

```
const w = window.open("", "_blank", …);
<h1>${title}</h1> … <h3>${p.name || "Apresentador"}</h3> … ${p.exact_speech.replace(/\n/g, "<br>")}
w.document.write(html);
```

POR QUE É EXPLORÁVEL
Uma fala ou título com <img src=x onerror=…> executa script na origem slideai.com.br quando o usuário clica em imprimir o roteiro.

CONDIÇÕES DE EXPLORAÇÃO
O painel só aparece no Editor do dono, então hoje é self-XSS. Passa a afetar outros usuários se o conteúdo vier de terceiros (duplicar ou importar apresentações, templates) ou de uma IA manipulada.

IMPACTO
Execução de script com acesso à sessão do usuário.

CORREÇÃO SUGERIDA
Escapar todos os campos (função escapeHtml) ou montar o documento com textContent.

BAIXA

A11 · og-preview insere título e descrição num bloco <script> JSON-LD sem escapar "<"

Categoria 5 · XSS / inputs sem tratamento · issue 8

EVIDÊNCIA
`supabase/functions/og-preview/index.ts:63-70`

```
<script type="application/ld+json">${JSON.stringify({ …, name: title, description, url, … })}</script>
```

POR QUE É EXPLORÁVEL
Um título publicado com </script><script>…</script> fecha o bloco e injeta script na página.

CONDIÇÕES DE EXPLORAÇÃO
A função hoje não está publicada (404 na verificação de 04/10) e o site não a referencia. Vira explorável se for publicada e servida como HTML.

IMPACTO
XSS armazenado na página de preview.

CORREÇÃO SUGERIDA
Serializar com JSON.stringify(...).replace(/</g, "\\u003c") ou remover o bloco JSON-LD.

INFORMATIVA

A15 · avatar_url aceita qualquer URL externa e é carregada por quem visita o perfil

Categoria 5 · XSS / inputs sem tratamento · issue 6

EVIDÊNCIA
`src/pages/PublicProfile.tsx:170`

```
<AvatarImage src={profile.avatar_url ?? undefined} alt={…} />
```

POR QUE É EXPLORÁVEL
Não executa script (img não roda javascript:), mas permite rastrear o IP e o navegador de quem visita o portfólio.

CONDIÇÕES DE EXPLORAÇÃO
Basta uma conta.

IMPACTO
Privacidade dos visitantes.

CORREÇÃO SUGERIDA
Aceitar só URLs do próprio Storage, validadas por CHECK ou trigger.

5. Recomendações priorizadas

P1 · Imediato

- Travar profiles.email (trigger) e resolver o pagamento por auth.users no cakto-webhook; índice único em lower(email). (A1)
- Rotacionar CAKTO_WEBHOOK_SECRET na Cakto e na Lovable, cadastrando o valor só pela tela de Secrets; confirmar que nenhum valor antigo é aceito. (A2, A3)

P2 · Antes de abrir para mais usuários

- Criar apresentações como não publicadas por padrão; leitura pública só por slug via RPC/view com colunas mínimas; listagem por dono respeitando can_view_portfolio; MCP filtrando por user_id. (A4)
- Proteger include_speeches, is_paid e presenters_* por trigger; cotas de IA por usuário e período; teto de slides em regenerate-speeches. (A5)

P3 · Próximo ciclo

- Suporte: forçar role='user' no INSERT do dono e restringir o UPDATE de conversas. (A6, A7)
- Storage: file_size_limit e allowed_mime_types no bucket avatars; avatar_url só do próprio Storage. (A8, A15)
- Revogar anon/authenticated de has_role e can_view_portfolio; reduzir o que get_profile_for_viewer devolve. (A9)
- Escapar HTML em PresenterNotesPanel e no JSON-LD do og-preview. (A10, A11)

P4 · Higiene contínua

- .env no .gitignore; papéis administrativos fora das migrações. (A12, A13)
- Visualizações por RPC com deduplicação. (A14)
- Adicionar gitleaks e um teste de RLS (pgTAP ou script) ao fluxo de PR.

6. ISSUES PARA O GITHUB

Texto completo de cada issue, pronto para copiar e colar. Cada bloco vai de --- ISSUE n --- a --- FIM ISSUE n ---: a linha "Título:" vai no campo de título e o resto no corpo. Achados relacionados foram agrupados para evitar issues repetidas. O mesmo conteúdo está em issues-github.md, sem quebras de linha, para colar com mais facilidade.

Nº	Severidade	Título	Achados
1	ALTA	[Segurança] E-mail do perfil editável desvia pagamentos da Cakto	A1
2	MÉDIA	[Segurança] Rotacionar o segredo do webhook da Cakto e eliminar as exposições	A2, A3
3	MÉDIA	[Segurança] Apresentações publicadas são listáveis por qualquer pessoa e furam o portfólio privado	A4
4	MÉDIA	[Segurança] Flags pagas e cotas de IA controladas pelo cliente	A5
5	BAIXA	[Segurança] Usuário forja mensagens da equipe e altera o estado dos próprios chamados	A6, A7
6	BAIXA	[Segurança] Avatares: sem limite de tipo/tamanho no bucket e URL externa livre	A8, A15
7	BAIXA	[Segurança] Superfícies anônimas: enumeração de papéis/acessos e visualizações infláveis	A9, A14
8	BAIXA	[Segurança] HTML montado sem escape no roteiro de falas e no JSON-LD do og-preview	A10, A11
9	INFORMATIVA	[Segurança] Higiene: .env versionado e papel admin concedido por migração	A12, A13

```
--- ISSUE 1 ---
Título: [Segurança] E-mail do perfil editável desvia pagamentos da Cakto
Labels: security, severidade:alta

## Descrição do problema

A coluna `profiles.email` pode ser alterada pelo próprio usuário: a política `profiles_update_own` não
restringe colunas e o trigger `protect_billing_columns` não inclui `email`. O `cakto-webhook` resolve o
dono da compra procurando primeiro esse e-mail em `profiles`. Um usuário que grave no próprio perfil o
e-mail de outra pessoa recebe o plano ou os créditos quando ela compra sem ter perfil com aquele e-mail
(compra antes do cadastro ou com outro e-mail).

## Por que é explorável

Com a chave anon (pública) e a própria sessão, supabase.from('profiles').update({ email:
'vitima@empresa.com' }).eq('id', meuId) passa pela RLS e pelo trigger. Quando a vítima compra com esse
e-mail, o webhook encontra o perfil do atacante e libera nele o plano ou os créditos. Condições: O
e-mail da compra não pode estar em outro perfil: com dois perfis iguais o maybeSingle falha e o webhook
cai em auth.users, que resolve a vítima. Vale para quem compra antes de criar a conta ou com outro
e-mail. É um ataque direcionado: o atacante precisa saber o e-mail com antecedência.

## Evidência

- `supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:16` — `CREATE POLICY
  "profiles_update_own" ON public.profiles FOR UPDATE USING (auth.uid() = id);`
- `supabase/migrations/20260827000346_68e8e98f-500b-4310-8345-f07ef89f505f.sql:206-224` —
  `protect_billing_columns()` não verifica `email`.
- `supabase/functions/cakto-webhook/index.ts:134-137` — `.from("profiles").select("id").ilike("email",
  escapeLike(email)).maybeSingle()` define o `userId`.
- Índice `profiles_email_lower_idx` não é único.
- `supabase/functions/_shared/lifecycle.ts:32` e `supabase/functions/send-email/index.ts:62,74,86`
  enviam e-mails para `profiles.email`.

## Impacto

Desvio de compras e assinaturas de terceiros, e e-mails da plataforma enviados para endereços escolhidos
pelo atacante.
```

```
## Sugestão de correção

- Adicionar `email` à lista de colunas travadas (ou sincronizar `profiles.email` a partir de
  `auth.users` por trigger).
- No `cakto-webhook`, resolver primeiro por `auth.users` (`find_user_id_by_email`) e só depois pelo
  perfil.
- Criar índice único em `lower(email)` e tratar duplicados existentes.

## Critérios de aceite

- [ ] `update profiles set email = ...` feito pelo cliente retorna erro 42501.
- [ ] Teste do webhook com dois perfis com o mesmo e-mail credita a conta de `auth.users`.
- [ ] Índice único em `lower(email)` criado em produção.
- [ ] Lifecycle e send-email usam o e-mail de `auth.users`.

_Achados do relatório de auditoria de 09/10/2026: A1._
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
Título: [Segurança] Rotacionar o segredo do webhook da Cakto e eliminar as exposições
Labels: security, severidade:média

## Descrição do problema

O segredo do webhook da Cakto foi exposto duas vezes:

1. O valor real em produção foi commitado em `supabase/functions/cakto-webhook/index.test.ts:13` (commit
`1cdc8b0`, 01/08/2026). O repositório é público, e o valor foi aceito até a rotação em 28/09/2026
(impressão `46925c8066f7`).
2. O valor atual (impressão `2bf40c2c9acd`) foi colado em texto puro no chat do projeto na Lovable em
29/09/2026.

Com o segredo, qualquer pessoa pode forjar eventos de pagamento.

## Por que é explorável

**A2** — Entre 01/08 e 28/09/2026, qualquer pessoa com o histórico podia enviar eventos falsos
(purchase_approved, subscription_created…) com o segredo válido e liberar planos ou créditos para
qualquer e-mail. Condições: Hoje o valor não é mais aceito: o segredo foi rotacionado e a lista esperada
só tem 2bf40c2c9acd. Forense: os 7 eventos de pagamento entre 01/08 e 29/09 não foram creditados a
nenhuma conta (user_id nulo).

**A3** — Quem tem acesso ao histórico do projeto (membros do workspace; visibilidade workspace_edit) ou
ao armazenamento do fornecedor pode forjar eventos de pagamento válidos. Condições: É preciso ter acesso
ao projeto na Lovable. O segredo continua válido.

## Evidência

- `git show 1cdc8b0:supabase/functions/cakto-webhook/index.test.ts` — linha 13 (`const SECRET =
  "[oculto]"`).
- `security_events` (cakto-webhook): impressão esperada `46925c8066f7` até 28/09/2026 e `2bf40c2c9acd`
  depois.
- Forense: nenhum dos 7 eventos de pagamento entre 01/08 e 29/09/2026 foi creditado a uma conta.

## Impacto

Liberação indevida de planos e créditos e reembolsos falsos.

## Sugestão de correção

- Gerar um novo segredo na Cakto e atualizar `CAKTO_WEBHOOK_SECRET` só pela tela de Secrets da Lovable.
- Conferir que a lista aceita contém apenas o novo valor.
- Usar só valores fictícios em testes e adicionar gitleaks ao fluxo de PR.
- (Opcional) Reescrever o histórico para remover o valor antigo.

## Critérios de aceite

- [ ] A impressão esperada em `security_events` muda para o novo valor.
- [ ] Um evento de teste da Cakto com o novo segredo é aceito; com os dois antigos, recebe 401.
- [ ] Workflow de varredura de segredos ativo no repositório.

_Achados do relatório de auditoria de 09/10/2026: A2, A3._
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
Título: [Segurança] Apresentações publicadas são listáveis por qualquer pessoa e furam o portfólio
privado
Labels: security, severidade:média

## Descrição do problema

A política `presentations_public_read` permite que anônimos leiam todas as colunas de todas as
apresentações publicadas, e toda apresentação gerada nasce com `is_published: true`. Com a chave anon é
possível listar todas as apresentações de um perfil privado (o `user_id` vem de
`get_profile_for_viewer`), incluindo `creative_brief` e `brand_identity`, ignorando
`can_view_portfolio`. A ferramenta `list_presentations` do MCP também devolve as apresentações
publicadas de todos os usuários.

## Por que é explorável

Com a chave anon: GET /rest/v1/presentations?select=*&is_published=eq.true&user_id=eq.<id> lista todas
as apresentações de um portfólio privado, incluindo creative_brief, brand_identity e presenters_names. A
verificação can_view_portfolio só existe na RPC get_portfolio_presentations. A ferramenta
list_presentations do MCP também mistura as apresentações publicadas de todos os usuários. Condições:
Basta a chave anon pública. Em produção, 50 de 52 apresentações estão publicadas, hoje todas de uma
única conta.

## Evidência

- `supabase/migrations/20260419011055_21b30c1d-4d93-4844-bbdf-d4be80e96700.sql:44` e `:66`
  (presentations_public_read, slides_public_read).
- `src/lib/legacyPersist.ts:88` e `supabase/functions/generate-presentation/persist.ts:82` —
  `is_published: true`.
- `supabase/functions/mcp/index.ts:81` / `src/lib/mcp/tools/list-presentations.ts:22-23` — consulta sem
  filtro por `user_id`.

## Impacto

Exposição do conteúdo e dos metadados de marca de todos os usuários e quebra do portfólio privado.

## Sugestão de correção

- Criar apresentações como não publicadas por padrão (o usuário publica explicitamente).
- Trocar a leitura pública direta por RPC ou view por slug, com colunas mínimas e sem listagem aberta.
- Fazer a listagem por dono respeitar `can_view_portfolio`.
- Filtrar por `user_id = ctx.getUserId()` no MCP.

## Critérios de aceite

- [ ] `GET /rest/v1/presentations?is_published=eq.true` com a chave anon não lista apresentações.
- [ ] O visualizador público por slug continua funcionando.
- [ ] Portfólio privado sem acesso aprovado não expõe apresentações por nenhuma rota.
- [ ] `list_presentations` devolve só as apresentações do usuário autenticado.

_Achados do relatório de auditoria de 09/10/2026: A4._
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
Título: [Segurança] Flags pagas e cotas de IA controladas pelo cliente
Labels: security, severidade:média

## Descrição do problema

`include_speeches` (recurso de +50 créditos) e `presenters_*` são gravados pelo navegador
(`legacyPersist.ts`), e a RLS permite alterá-los. O `regenerate-speeches` confia em `include_speeches` e
não limita o número de slides. As cotas do `chat-editor` e do `regenerate-speeches` são contadas por
apresentação, uma unidade que o cliente cria livremente pela API.

## Por que é explorável

Um usuário com plano ativo (ou bônus) cria pela API apresentações com include_speeches=true (sem pagar
os +50 créditos) e com quantos slides quiser, depois chama regenerate-speeches até 5 vezes em cada uma.
Cada apresentação nova também zera a cota do chat-editor, cujo limite real é o rate limit de 40/h.
Condições: Exige conta autenticada com direito de uso (can_user_generate com custo 0).

## Evidência
```

```
- `src/lib/legacyPersist.ts:86-90` — insert com `include_speeches: form.includeSpeeches`.
- `supabase/migrations/20260927000100_platform_review_fixes.sql:233-251` — o trigger protege só os
  contadores.
- `supabase/functions/regenerate-speeches/index.ts:16,67,88` — 5 regenerações por apresentação, confia
  no flag, escopo `all` sem teto.
- `supabase/functions/chat-editor/index.ts:198-211` — cota por apresentação.

## Impacto

Custo de IA sem cobrança e contorno do preço das falas.

## Sugestão de correção

- Travar `include_speeches`, `is_paid` e `presenters_*` por trigger (só service role grava).
- Contar as cotas de edição e de regeneração por usuário e período.
- Limitar o número de slides enviados ao modelo em `regenerate-speeches`.

## Critérios de aceite

- [ ] O cliente não consegue mudar `include_speeches` (erro 42501).
- [ ] Criar apresentações novas não renova a cota de IA.
- [ ] `regenerate-speeches` recusa decks acima do limite de slides.

_Achados do relatório de auditoria de 09/10/2026: A5._
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
Título: [Segurança] Usuário forja mensagens da equipe e altera o estado dos próprios chamados
Labels: security, severidade:baixa

## Descrição do problema

`msg_own_insert` não restringe `role` nem `metadata`: o dono do chamado insere mensagens
`role=assistant` com `metadata.source=human_agent`, o mesmo formato da resposta humana.
`conv_own_update` permite alterar `state`, `escalated_at`, `resolved_by_human`, `reopen_count` e
`rating`.

## Por que é explorável

**A6** — Via PostgREST, o dono do chamado insere role="assistant" e metadata.source="human_agent" (por
exemplo, "reembolso aprovado"). A mensagem aparece como resposta da equipe e entra no contexto do
modelo. Condições: Afeta só o próprio chamado.

**A7** — O usuário grava state='escalated', escalated_at, resolved_by_human, reopen_count ou rating
direto pela API. Condições: Afeta só as conversas do próprio usuário.

## Evidência

- `supabase/migrations/20260716013934_90aa6f03-b7f6-4313-b724-57a99fea27f7.sql:135-137` (msg_own_insert)
  e `:109-110` (conv_own_update).
- `src/pages/AdminSupport.tsx:154-158` — formato da resposta humana.
- `supabase/functions/support-chat/index.ts:243-245` — mensagens assistant do histórico vão ao modelo.

## Impacto

Engenharia social contra o suporte, indução do assistente e fura-fila ou métricas falsas.

## Sugestão de correção

- Exigir `role = 'user'` e `metadata` vazia no INSERT do dono (WITH CHECK).
- Remover o UPDATE do dono em `support_conversations` (as transições passam pela edge function) ou
  travar as colunas de staff por trigger.

## Critérios de aceite

- [ ] INSERT do dono com `role = 'assistant'` falha.
- [ ] UPDATE do dono em `state` falha; o fluxo do support-chat continua funcionando.

_Achados do relatório de auditoria de 09/10/2026: A6, A7._
--- FIM ISSUE 5 ---
```

```
--- ISSUE 6 ---
Título: [Segurança] Avatares: sem limite de tipo/tamanho no bucket e URL externa livre
Labels: security, severidade:baixa

## Descrição do problema

O bucket `avatars` não define `file_size_limit` nem `allowed_mime_types`. A validação existe só no
navegador (`avatarUpload.ts`). `profiles.avatar_url` aceita qualquer URL externa, que é carregada por
quem visita o portfólio.

## Por que é explorável

**A8** — Chamando a API do Storage direto, o usuário sobe arquivos de qualquer tipo e tamanho (HTML,
SVG, binários grandes) na própria pasta. Eles ficam acessíveis por URL assinada de 10 anos e podem ser
lidos por anônimos. Condições: Basta uma conta autenticada.

**A15** — Não executa script (img não roda javascript:), mas permite rastrear o IP e o navegador de quem
visita o portfólio. Condições: Basta uma conta.

## Evidência

- `storage.buckets`: avatars com `file_size_limit = NULL` e `allowed_mime_types = NULL`.
- `src/lib/avatarUpload.ts:3-4,14-15,22` — validação só no cliente; `contentType: file.type`.
- `src/pages/PublicProfile.tsx:170` — `<AvatarImage src={profile.avatar_url} />`.

## Impacto

Hospedagem de conteúdo arbitrário no storage do projeto e rastreamento de visitantes.

## Sugestão de correção

- Configurar `file_size_limit = 4 MB` e `allowed_mime_types = png, jpeg, webp, gif` no bucket.
- Aceitar em `avatar_url` só URLs do próprio Storage (CHECK ou trigger).

## Critérios de aceite

- [ ] Upload de 10 MB ou de `text/html` pela API é recusado.
- [ ] Gravar `avatar_url` externa falha.

_Achados do relatório de auditoria de 09/10/2026: A8, A15._
--- FIM ISSUE 6 ---
```

```
--- ISSUE 7 ---
Título: [Segurança] Superfícies anônimas: enumeração de papéis/acessos e visualizações infláveis
Labels: security, severidade:baixa

## Descrição do problema

`has_role(uuid, app_role)` e `can_view_portfolio(_owner, _viewer)` podem ser executadas por anônimos com
IDs arbitrários (revelam contas admin e acessos aprovados), e `get_profile_for_viewer` devolve o `id` de
qualquer username. `views_published_insert` permite inserir visualizações em massa, inflando
`view_count`.

## Por que é explorável

**A9** — Com IDs obtidos em presentations.user_id (público) ou em get_profile_for_viewer, um anônimo
descobre quais contas são admin ou developer (has_role) e quem tem acesso aprovado a um portfólio
privado (can_view_portfolio com _viewer arbitrário). Condições: Basta a chave anon.

**A14** — Um script com a chave anon insere linhas em massa, e o trigger bump_presentation_view_count
infla view_count. Condições: Basta a chave anon.

## Evidência

- `supabase/migrations/20260806000729_867c7dc3-03da-4aea-aa55-d62f0694eb74.sql:38` — GRANT de has_role a
  anon.
- `supabase/migrations/20260814235606_fe2269e2-9079-4ffe-8ae8-2adfe259f9fa.sql:41,61-65,120-121`.
- `supabase/migrations/20260806000646_a3684224-f5cb-4467-ab32-3685b698016a.sql:23` —
  views_published_insert.

## Impacto

Enumeração de alvos privilegiados, vazamento de relações e métricas falsas.

## Sugestão de correção
```

```
- Revogar EXECUTE de anon e authenticated em `has_role` e `can_view_portfolio` (as políticas continuam
  funcionando como definer).
- Fazer `can_view_portfolio` usar só `auth.uid()`; `get_profile_for_viewer` sem `id` quando não houver
  acesso.
- Registrar visualizações por RPC com deduplicação.

## Critérios de aceite

- [ ] `rpc('has_role', …)` com a chave anon retorna erro de permissão.
- [ ] Inserir 100 visualizações seguidas da mesma origem conta uma.

_Achados do relatório de auditoria de 09/10/2026: A9, A14._
--- FIM ISSUE 7 ---
```

```
--- ISSUE 8 ---
Título: [Segurança] HTML montado sem escape no roteiro de falas e no JSON-LD do og-preview
Labels: security, severidade:baixa

## Descrição do problema

`PresenterNotesPanel.printScript` interpola título, headlines, nomes e falas direto em HTML escrito com
`document.write` numa janela do mesmo domínio. O `og-preview` insere `JSON.stringify(...)` dentro de
`<script>` sem neutralizar `</script>`.

## Por que é explorável

**A10** — Uma fala ou título com <img src=x onerror=…> executa script na origem slideai.com.br quando o
usuário clica em imprimir o roteiro. Condições: O painel só aparece no Editor do dono, então hoje é
self-XSS. Passa a afetar outros usuários se o conteúdo vier de terceiros (duplicar ou importar
apresentações, templates) ou de uma IA manipulada.

**A11** — Um título publicado com </script><script>…</script> fecha o bloco e injeta script na página.
Condições: A função hoje não está publicada (404 na verificação de 04/10) e o site não a referencia.
Vira explorável se for publicada e servida como HTML.

## Evidência

- `src/components/PresenterNotesPanel.tsx:86-114`.
- `supabase/functions/og-preview/index.ts:63-70`.

## Impacto

Execução de script na origem do app (hoje self-XSS) e XSS armazenado no preview, caso a função seja
publicada.

## Sugestão de correção

- Escapar todos os campos com `escapeHtml` (ou usar `textContent`) no roteiro.
- No JSON-LD, `JSON.stringify(obj).replace(/</g, "\\u003c")`.

## Critérios de aceite

- [ ] Fala contendo `<img src=x onerror=alert(1)>` aparece como texto no roteiro impresso.
- [ ] Título com `</script>` não quebra o HTML do og-preview (teste unitário).

_Achados do relatório de auditoria de 09/10/2026: A10, A11._
--- FIM ISSUE 8 ---
```

```
--- ISSUE 9 ---
Título: [Segurança] Higiene: .env versionado e papel admin concedido por migração
Labels: security, severidade:informativa

## Descrição do problema

O `.env` com a chave anon (pública por design) está versionado e fora do `.gitignore`. Uma migração
concede `admin` e `developer` a um UUID fixo, expondo o ID da conta administradora num repositório
público.

## Por que é explorável

**A12** — Não é um segredo, porque a segurança vem da RLS. O risco é alguém acrescentar uma chave real
(service role, OpenAI) a esse arquivo já rastreado.

**A13** — Expõe o ID da conta administradora, facilitando ataques direcionados junto com A9. Todo
ambiente criado a partir do repositório herda esse admin.
```

```
## Evidência

- `.env:1-3` e `.env.example:1-6`.
- `supabase/migrations/20260517133059_9502e9f2-1012-4cec-8021-84c751280b72.sql:38-41`.

## Impacto

Risco de uma chave real ir parar num arquivo já rastreado e alvo administrativo conhecido.

## Sugestão de correção

- Adicionar `.env` ao `.gitignore` (git rm --cached .env) e manter `.env.example`.
- Documentar a concessão de papéis como procedimento manual, fora das migrações.

## Critérios de aceite

- [ ] `git ls-files .env` vazio.
- [ ] Nenhuma migração nova concede papel a UUID fixo.

_Achados do relatório de auditoria de 09/10/2026: A12, A13._
--- FIM ISSUE 9 ---
```